

Adaptive Priority Arbitration for Stateful Distributed Pipelines

Robert Guy

Abstract

Modern distributed pipelines frequently process continuous streams of updates that contribute to evolving shared state across multiple system components. Examples include telemetry processing platforms, distributed sensor fusion systems, real-time analytics pipelines, and online inference services. In these environments, resource arbitration decisions influence not only throughput but also state freshness, latency variance, and the correctness of downstream computations. Traditional scheduling approaches such as FIFO, static-priority queues, and fair-share policies are often insufficient because they treat jobs as independent tasks rather than components of stateful distributed workflows with ordering constraints and latency-sensitive updates.

This paper examines resource arbitration as a distinct systems problem in stateful distributed pipelines. We introduce an adaptive priority arbitration framework that combines static importance with runtime signals including mean latency, latency variance, backlog growth, and state freshness. The framework also incorporates inheritance-style priority escalation to mitigate cross-stage priority inversion in multi-stage pipelines. Rather than advocating a single scheduling rule, we define a policy taxonomy that includes ordered, latency-sensitive, variance-sensitive, inheritance-based, and fair-share arbitration models.

We evaluate the proposed approach using a simulation-based framework that compares adaptive arbitration policies against traditional scheduling baselines under mixed-priority and bursty workloads. Our results show that adaptive policies incorporating latency and freshness signals substantially reduce tail latency while maintaining balanced throughput and preserving ordering guarantees, suggesting that adaptive arbitration provides a practical alternative to rigid scheduling policies in stateful distributed systems.

1 Introduction

Modern distributed pipelines increasingly operate under competing latency, throughput, and correctness constraints. Examples include telemetry processing systems, distributed sensor fusion pipelines, online control systems, and inference-serving stacks. In these environments, jobs do not simply consume compute in isolation; they often contribute to evolving shared or derived state, interact across multiple pipeline stages, and contend for limited resources such as CPU time, queue capacity, network bandwidth, or accelerator access. As a result, resource arbitration decisions can directly influence not only throughput, but also state freshness, tail latency, and end-to-end correctness [1].

Traditional scheduling approaches are often poorly matched to this setting. First-in, first-out scheduling is simple and predictable, but it treats all work as equally urgent even when some streams are latency-sensitive or semantically stale. Static-priority scheduling improves responsiveness for high-priority work, but it does not adapt when workload conditions change and can lead to starvation or instability under bursty arrivals. Fair-share approaches improve utilization and prevent starvation, yet they may ignore ordering constraints, deadline risk, and the cost of allowing stateful updates to lag behind real time. More broadly, most existing

scheduling policies are designed either for local threads and processes or for large-scale stateless cluster workloads; they do not directly address the arbitration needs of *stateful distributed pipelines* in which freshness, variance, and cross-stage blocking all matter.

This paper argues that resource arbitration for stateful distributed pipelines should be treated as a distinct systems problem. In these systems, the schedulable object is often not just a thread, but a job, message, or update stream whose value depends on timing and ordering. A delayed job may increase downstream queuing, degrade the freshness of shared state, or block higher-priority work in later stages of the pipeline [2, 4]. Consequently, effective arbitration requires more than static priorities or average-latency optimization. It requires policies that account for dynamic workload behavior, including latency variance, backlog growth, staleness, and blocking relationships between pipeline stages.

To address this problem, we propose an adaptive priority arbitration framework for stateful distributed pipelines. The framework combines base priority with runtime signals such as mean latency, latency variance, queue backlog, and state freshness. It also supports inheritance-style escalation mechanisms to mitigate priority inversion across dependent stages. Rather than advocating a single fixed scheduling rule, we define a family of policy classes that include ordered, latency-sensitive, variance-sensitive, inheritance-based and fair-share modes that can be selected or combined depending on application requirements. This framing allows arbitration behavior to reflect the semantics of the workload rather than relying on a one-size-fits-all queue discipline. The central claim of this paper is that arbitration policies for stateful distributed pipelines should jointly optimize responsiveness and state quality. In particular, we hypothesize that adaptive policies that incorporate latency, variance, and freshness signals can reduce tail latency and improve state freshness relative to FIFO and static-priority baselines, while still preserving ordering guarantees where required. We evaluate this claim using a simulation based model of a multi-stage distributed pipeline with mixed-priority and bursty workloads.

This paper makes the following contributions:

- We identify resource arbitration in stateful distributed pipelines as a distinct systems problem characterized by latency sensitivity, ordering constraints, state freshness, and cross-stage blocking.
- We define a policy taxonomy for arbitration in these systems, including ordered, latency-sensitive, variance-sensitive, inheritance-based, and fair-share policies.
- We propose an adaptive priority model that combines static importance with runtime signals such as latency, variance, backlog, and freshness.
- We outline an evaluation methodology and simulation framework for comparing adaptive arbitration policies with traditional scheduling baselines.

The remainder of this paper is organized as follows. Section 2 presents background and motivating challenges in resource arbitration for stateful distributed pipelines. Section 3 introduces the system model and workload assumptions used throughout the paper. Section 4 describes the proposed arbitration policy framework and adaptive priority model. Section 5 outlines the evaluation methodology and simulation environment used to compare policies. Section 6 discusses expected results and implications for real-world distributed systems.

2 Background and Motivation

Modern distributed systems frequently organize computation as multi-stage pipelines that process continuous streams of updates. Examples include telemetry processing platforms, distributed sensor fusion systems, online analytics pipelines, and machine learning inference services. In these systems, updates flow through a sequence of stages that perform filtering transformation, aggregation and state updates. Each stage consumes and produces messages while competing for shared system resources such as CPU time, queue capacity, network bandwidth, or accelerator access.

Unlike traditional batch workloads, the jobs processed by these pipelines often represent **state updates rather than independent computations**. An update may modify a shared state representation, trigger downstream processing, or contribute to an evolving model of the environment. As a result, scheduling decisions affect not only throughput but also properties such as state freshness, latency variance, and the ordering of updates. Delays or reordering can cause downstream stages to operate on stale information or temporarily inconsistent views of the system state.

Resource arbitration in these systems is therefore more complex than conventional task scheduling. Many widely used policies, such as first-in-first-out (FIFO) queues, treat all incoming work as equally urgent [5]. While FIFO is simple and predictable, it does not account for differences in latency sensitivity or semantic importance between update streams. For example, a high-rate telemetry source may generate bursts of updates that fill processing queues, delaying more critical updates from other sources. In such cases, FIFO scheduling can significantly increase tail latency and degrade the freshness of the system’s shared state.

Static-priority scheduling offers a partial improvement by allowing certain streams or tasks to receive preferential treatment. However, static priorities are often insufficient in dynamic distributed pipelines. Workload characteristics such as arrival rates, queue backlogs, and latency variance can change rapidly over time. Fixed priorities may therefore lead to starvation, unstable queue growth, or inefficient resource utilization under bursty or mixed-priority workloads.

Another challenge arises from **cross-stage dependencies** within pipelines. An update that is delayed in one stage can indirectly delay higher-priority work in downstream stages, creating forms of priority inversion similar to those observed in operating system schedulers [3]. Traditional queue-based scheduling approaches do not explicitly address these pipeline-level interactions.

These observations motivate the need for arbitration policies that are aware of both **runtime workload behavior** and **the semantics of state updates** flowing through distributed pipelines. Rather than relying on a single scheduling rule, it is useful to consider a broader design space of policies that incorporate latency signals, backlog information, and ordering constraints when deciding which work to execute next [6].

3 System Model

We model the target system as a **stateful distributed pipeline** consisting of multiple processing stages connected by message queues. Jobs are generated by one or more producers, flow through a sequence of intermediate stages, and may update or contribute to a shared state representation used by downstream consumers. Examples include telemetry processing systems, distributed sensor fusion pipelines, and online inference-serving architectures. The key characteristic of these systems is that jobs are not purely independent computations; instead they

often affect evolving shared or derived state whose usefulness depends on both correctness and timelines.

Formally, let the pipeline consist of stages

$$P = \{p_1, p_2, \dots, p_n\},$$

where each stage p_i is associated with a queue Q_i and a finite processing capacity. Jobs arrive at the pipeline from one or more input streams

$$S = \{s_1, s_2, \dots, s_m\}.$$

Each job j belongs to some stream s_k , arrives at time a_j , and carries a base priority b_j . A job may also carry metadata such as a deadline, an ordering constraint, or a state update timestamp. At each stage, the scheduler must decide which queued job to dispatch next given limited processing resources.

We treat a job as the fundamental **schedulable object**. A job may represent a telemetry update, a state synchronization message, an inference request, or another unit of work that progresses through the pipeline. Each job is characterized by:

$$j = (s_j, a_j, c_j, b_j, t_j),$$

where s_j is the originating stream, a_j is the arrival time, c_j is the service cost or processing demand, b_j is the base priority, and t_j is the timestamp associated with the state or event represented by the job. Additional fields may encode ordering requirements or dependency relationships.

Resource arbitration occurs because each stage has limited service capacity. At a given time, multiple jobs may compete for:

- CPU execution slots,
- queue capacity,
- network transmission opportunities, or
- accelerator access in heterogeneous settings.

The scheduler at each stage must choose a dispatch order that balances throughput, responsiveness, and correctness-related constraints [2, 4].

A key feature of the model is that jobs may contribute to an evolving shared state. Let $x(t)$ denote the most recent state visible to downstream consumers at time t . Because jobs may be delayed in queues or at intermediate stages, the state available to the system may lag behind the most recent input. We therefore define **state freshness** informally as the age of the newest incorporated update relative to the current time. If the latest applied update at time t carries timestamp t^* , then freshness lag can be expressed as

$$F(t) = t - t^*$$

Smaller values of $F(t)$ indicate fresher state, while larger values indicate that the pipeline is operating on increasingly stale information.

In addition to state freshness, we track standard latency metrics. For a job j , let completion time be d_j . End-to-end latency is then

$$L_j = d_j - a_j.$$

Across a stream or workload class, we are interested not only in mean latency but also in **tail latency** and **latency variance**, since distributed pipelines are often sensitive to occasionally large delays even when average performance appears acceptable.

The model also captures **ordering constraints**. Some streams require updates to be processed in first-in-first-out order to preserve application semantics, while others permit reordering when doing so improves freshness or responsiveness. We therefore treat ordering policy as part of the arbitration problem rather than as a fixed global assumption [6].

Finally, the model admits **cross-stage dependency effects**. A job delayed in an upstream stage may block or indirectly delay higher-priority work downstream, creating a distributed form of priority inversion. This motivates arbitration mechanisms that can react not only to local queue state but also to pipeline-level signals such as backlog growth, downstream blocking, and state staleness.

Under this model, the goal of arbitration is not merely to maximize throughput. Instead, the system must balance several competing objectives: low average latency, bounded tail latency, reduced variance, preservation of required ordering constraints, prevention of starvation, and timely incorporation of updates into shared state. The following sections build on this model to define a family of arbitration policies and an adaptive priority framework for stateful distributed pipelines.

4 Arbitration Policy Framework

Distributed pipelines operate under diverse workload conditions and application constraints making it difficult for a single scheduling rule to perform well across all scenarios [3]. Rather than prescribing a fixed arbitration strategy, we instead define a policy framework that captures several classes of scheduling behavior commonly required in stateful distributed pipelines. These policy classes reflect different trade-offs between responsiveness, fairness, ordering guarantees, and state freshness.

The framework treats arbitration as the problem of selecting the next job to execute from a queue of competing work items. Each job carries metadata describing its origin stream, priority signals, and ordering requirements. Arbitration policies determine how this metadata is interpreted when selecting the next job for execution. The policies described below represent common behavioral modes that arise in distributed pipeline systems.

4.1 Ordered Policies

Some workloads require updates to be processed in strict arrival order. For example, state synchronization streams or causally ordered update logs may rely on first-in-first-out (FIFO) processing to preserve correctness. Ordered policies enforce this constraint by selecting jobs strictly according to arrival order within each stream. While this approach preserves ordering semantics, it can degrade responsiveness when bursts of low-priority work delay more critical updates.

4.2 Latency-Sensitive Policies

In many real-time or soft real-time systems, certain update streams are more latency-sensitive than others. Latency-sensitive policies prioritize jobs that are approaching latency thresholds or whose delays may negatively affect downstream components. Such policies aim to minimize response time for critical updates even under bursty workloads.

4.3 Variance-Sensitive Policies

Distributed pipelines often experience unstable performance due to bursts of work or uneven queue growth. Variance-sensitive policies attempt to reduce latency instability by prioritizing streams or jobs that exhibit unusually high latency variance. By stabilizing service times across streams, these policies can reduce jitter and improve overall system predictability.

4.4 Inheritance-Based Policies

Pipeline stages may introduce indirect dependencies between jobs. A job delayed in one stage can block downstream processing of higher-priority updates, producing a distributed form of priority inversion. Inheritance-based policies mitigate this effect by allowing blocked jobs to temporarily inherit higher priority from dependent work. This mechanism is conceptually similar to priority inheritance in operating system schedulers.

4.5 Fair-Share Policies

While responsiveness is important for latency-sensitive streams, distributed systems must also avoid starvation of lower-priority work. Fair-share policies allocate processing opportunities across streams in a way that ensures long-term progress for all workloads. Such policies are particularly useful for background processing tasks that are not latency-critical but must eventually complete.

These policy classes are **not mutually exclusive**. In practice, arbitration behavior may combine several of these principles depending on system requirements. For example, a pipeline may enforce ordering within individual streams while still prioritizing latency-sensitive updates across streams. Similarly, fairness constraints may operate alongside inheritance mechanisms to prevent persistent priority inversion.

The next section introduces an **adaptive priority model** that provides a unified mechanism for implementing these policy behaviors using runtime signals such as latency, queue backlog, and state freshness.

5 Adaptive Priority Model

The policy framework described in the previous section defines the behavioral modes required in stateful distributed pipelines. We now introduce an **adaptive priority model** that provides a unified mechanism for implementing these modes. The core idea is to compute a dynamic priority score for each schedulable job using both static priority metadata and runtime signals derived from observed system behavior.

Let j denote a job waiting for service at some pipeline stage. We define its effective priority score as

$$\Pi(j) = w_b B(j) + w_l L(j) + w_v V(j) + w_q Q(j) + w_f F(j) + w_i I(j),$$

where:

- $B(j)$ is the base priority assigned to the job or stream,
- $L(j)$ is a latency-related term,
- $V(j)$ captures latency variance or instability,
- $Q(j)$ represents queue backlog or congestion,

- $F(j)$ represents state freshness pressure or staleness,
- $I(j)$ represents inheritance-based escalation due to downstream blocking,
- and $w_b, w_l, w_v, w_q, w_f, w_i$ are nonnegative weights.

The scheduler uses $\prod(j)$ to rank competing jobs at a stage. Larger scores correspond to higher urgency. The purpose of this formulation is not to claim that one fixed weighted sum is optimal for all systems, but to provide a general mechanism through which different policy behaviors can be expressed and compared.

5.1 Base Priority

The base priority term $B(j)$ represents static importance assigned by the application or system configuration. For example, control-plane updates, safety-critical telemetry, or user-facing requests, may receive higher base priority than background analytics or archival tasks. If arbitration depended only on $B(j)$, the scheduler would reduce to a static-priority policy.

5.2 Latency Term

The latency term $L(j)$ captures urgency induced by observed waiting time or deadline pressure. One simple choice is to set $L(j)$ proportional to the time the job has already spent in the system. Another possibility is to define $L(j)$ in terms of remaining slack relative to a target latency bound. In both cases, the goal is to increase urgency for jobs that are in danger of missing responsiveness objectives.

5.3 Variance Term

Mean latency alone is often insufficient for characterizing pipeline quality. Distributed pipelines may exhibit acceptable average latency while still suffering from large occasional delays. The term $V(j)$ captures instability in stream behavior, for example by measuring the recent variance of latency within the stream to which j belongs. This term enables the scheduler to react to jitter and protect streams whose performance is becoming unstable.

5.4 Queue Backlog Term

The backlog term $Q(j)$ reflects local congestion. A stream with growing backlog may require increased service priority to avoid runaway queue growth or downstream starvation. This term allows the scheduler to respond to bursty arrivals and helps differentiate isolated latency spikes from persistent overload.

5.5 Freshness Term

In stateful pipelines, some jobs are more important because they reduce the staleness of the system’s shared state. The freshness term $F(j)$ captures this effect. Intuitively, updates whose absence causes the visible state to lag further behind the current environment should receive increased urgency. This term is especially relevant for telemetry pipelines and state synchronization systems in which stale state may be less useful than timely approximate state.

5.6 Inheritance Term

The inheritance term $I(j)$ models distributed priority inversion. If a low-priority job in one stage blocks the progress of high-priority downstream work, the blocked dependency may temporarily increase the effective priority of the upstream job. This is analogous to priority inheritance in operating systems, but here the dependency graph may span multiple pipeline stages rather than a single lock or critical section.

The adaptive priority model is intentionally modular. By adjusting the weights or disabling specific terms, the system can realize different arbitration behaviors. For example:

- an ordered policy corresponds to prioritizing arrival order while suppressing most adaptive terms,
- a latency-sensitive policy emphasizes $L(j)$,
- a variance-sensitive policy emphasizes $V(j)$,
- a fairness-oriented policy may reduce the influence of static base priority and incorporate queue balancing,
- and an inheritance-based policy activates $I(j)$ when downstream blocking is detected.

This formulation also makes trade-offs explicit. Increasing the influence of freshness or latency terms may improve responsiveness for critical streams, but it can also reduce fairness for background work. Similarly, aggressive inheritance can reduce downstream blocking but may increase short-term priority volatility. The evaluation section therefore treats the adaptive priority model not as a single scheduler, but as a parameterized design space whose behaviors can be explored under controlled workloads.

In practice, the signals used in $\Pi(j)$ may be normalized to ensure comparability across streams and stages. For example, latency and queue depth can be rescaled relative to recent workload statistics, while freshness pressure may be normalized against the maximum observed state lag. These implementation details do not change the conceptual role of each term, but they are important for producing stable behavior in concrete systems.

The next section describes the evaluation methodology used to compare these arbitration strategies under mixed-priority, bursty, and state-sensitive workloads.

6 Evaluation Methodology

To evaluate the proposed arbitration framework, we consider a simulation-based model of a multi-stage distributed pipeline. The goal of the evaluation is to determine whether adaptive priority policies improve responsiveness and state quality relative to standard scheduling strategies under realistic workload conditions. In particular, we are interested in the impact of arbitration policy on mean latency, tail latency, latency variance, queue growth, starvation, and state freshness.

6.1 Simulation Environment

We model the system as a sequence of pipeline stages connected by finite queues. Jobs are generated by one or more input streams and traverse the pipeline until completion. Each stage has limited processing capacity and applies a configurable arbitration policy when selecting the

next job for service. The simulation operates in discrete event time, allowing job arrivals, service completions, queuing delays, and blocking effects to be modeled explicitly.

The simulated pipeline includes producers, intermediate processing stages, and a state aggregation stage that materializes the current visible system state. Each job is associated with a stream identifier, arrival time, service cost, and base priority. Additional metadata may include timestamps used to measure freshness lag and dependency information used to model inheritance-based priority escalation.

For the experiments reported in this paper, the adaptive scheduler used a fixed weighted scoring configuration with $w_b = 0.8$, $w_l = 0.02$, $w_q = 0.20$, $w_f = 0.5$, and $w_v = 0.1$, corresponding to base priority, latency pressure, queue backlog, freshness pressure, and latency variance, respectively. These values were selected empirically to illustrate the trade-off between freshness sensitivity and tail-latency stability. We do not claim that this configuration is universally optimal; rather, it represents a reasonable operating point for comparing adaptive arbitration against FIFO and static-priority baselines.

Weight	Value
w_b	0.8
w_l	0.02
w_q	0.20
w_f	0.5
w_v	0.1

Table 1: Adaptive scheduler weights used in the reported experiments.

6.2 Workload Classes

We evaluate arbitration policies under several workload classes designed to reflect common behaviors in real distributed systems.

Steady high-priority streams. These streams represent latency-sensitive updates such as control-plane messages or real-time telemetry. They arrive at relatively stable rates and are intended to test responsiveness under normal operating conditions.

Bursty streams. These streams generate occasional surges of work and model behavior such as sudden telemetry bursts, synchronized update storms, or event-driven workload spikes. They are used to evaluate queue buildup, latency instability, and scheduler adaptability.

Background streams. These streams represent lower-priority work such as logging, analytics, or archival processing. They are useful for measuring fairness and starvation behavior under adaptive arbitration.

Mixed workloads. In addition to evaluating each stream type independently, we consider mixed-priority scenarios in which steady, bursty, and backgrounds share the same pipeline resources.

6.3 Scheduling Baselines

We compare the proposed adaptive policies against several common scheduling baselines.

FIFO. Jobs are processed strictly in arrival order. This serves as the simplest baseline and provides a point of comparison for all adaptive policies.

Static Priority. Jobs are scheduled according to a fixed base priority. This baseline captures systems that distinguish between high and low-importance work but do not adapt to changing runtime conditions.

Round Robin or Fair Share. Jobs are selected to ensure balanced long-term progress across streams. This baseline is useful for evaluating starvation prevention and fairness trade-offs.

Adaptive Priority Policies. These include latency-sensitive, variance-sensitive, freshness-aware, and inheritance-based variants of the proposed model, as well as combined policies using multiple priority terms simultaneously.

6.4 Metrics

We evaluate the policies using the following metrics.

Mean latency. For each job j , latency is defined as $L_j = d_j - a_j$, where a_j is arrival time and d_j is completion time. We report average latency across jobs and streams.

Tail latency. We report high-percentile latency, such as the 95th or the 99th percentile, in order to capture rare but significant delays that affect system responsiveness.

Latency variance. We measure variance in job latency within each stream to quantify instability and jitter.

Throughput. We report completed jobs per unit time in order to evaluate whether adaptive policies trade off responsiveness against total work completed.

Starvation rate. We measure the fraction of jobs or streams that experience excessive delay relative to their expected service rate.

State freshness. Let t^* be the timestamp of the most recent update incorporated into visible state at time t . Freshness lag is defined as $F(t) = t - t^*$. We report average and worst-case freshness lag to quantify how well the system maintains current state under load.

6.5 Experimental Scenarios

The simulation is designed to study several recurring scenarios in distributed pipelines.

Normal load. The system operates below saturation, allowing comparison of arbitration behavior under stable conditions.

Burst amplification. One or more streams generate short bursts of high arrival rates, testing how well policies manage backlog growth and protect latency-sensitive work.

Mixed-priority contention. High-priority and background streams compete for the same resources, allowing measurement of responsiveness and starvation trade-offs.

Cross-stage blocking. Jobs in an upstream stage delay progress of dependent downstream work, enabling evaluation of inheritance-based escalation.

6.6 Hypotheses

Our evaluation is designed to test the following hypotheses:

- Adaptive arbitration policies reduce tail latency relative to FIFO and static-priority scheduling under bursty workloads.
- Variance-sensitive policies reduce latency instability compared with policies that optimize only average delay.
- Freshness-aware policies reduce state staleness for update streams whose timeliness is more important than strict arrival order.
- Inheritance-based escalation reduces the effects of distributed priority inversion in multi-stage pipelines.

Taken together, these experiments allow us to evaluate the adaptive priority model as a parameterized scheduling framework rather than as a single fixed scheduler. The next section presents the experimental results and analyzes the trade-offs between responsiveness, fairness, and state freshness across policies.

7 Experimental Results

To evaluate the effectiveness of the proposed adaptive arbitration strategy, we conducted a simulation comparing three scheduling policies: FIFO scheduling, static priority scheduling, and the proposed adaptive priority scheduler. The goal of the experiment was to evaluate the trade-offs between latency stability and state freshness when processing heterogeneous update streams.

The simulated system processes three distinct update streams representing different workload characteristics: **burst**, **control**, and **background** traffic. Each stream produces update events with different arrival patterns and priorities. Burst traffic represents intermittent spikes of activity, control traffic represents time-sensitive updates requiring fresh state information, and background traffic represents lower-priority updates with less strict freshness requirements. All experiments were conducted using the same workload parameters to ensure a fair comparison across scheduling strategies.

For each policy, the simulator collected multiple performance metrics including mean latency, tail latency (P95 and P99), latency standard deviation, throughput, and mean freshness lag. Latency was measured as the time between job arrival and completion, while freshness lag measures the difference between the timestamp of the most recently applied update and the timestamp of the update being processed.

7.1 Latency Behavior

Figure 1 shows the **mean latency** observed for each scheduling policy. Static priority scheduling produces the lowest mean latency because high-priority updates are processed immediately, reducing average waiting time for those tasks. FIFO and adaptive priority scheduling show slightly higher mean latency, as these strategies balance work more evenly across streams. However, mean latency alone does not capture the full behavior of the system. Tail latency is often more important in real-time or state synchronization systems where extreme delays can degrade system consistency.

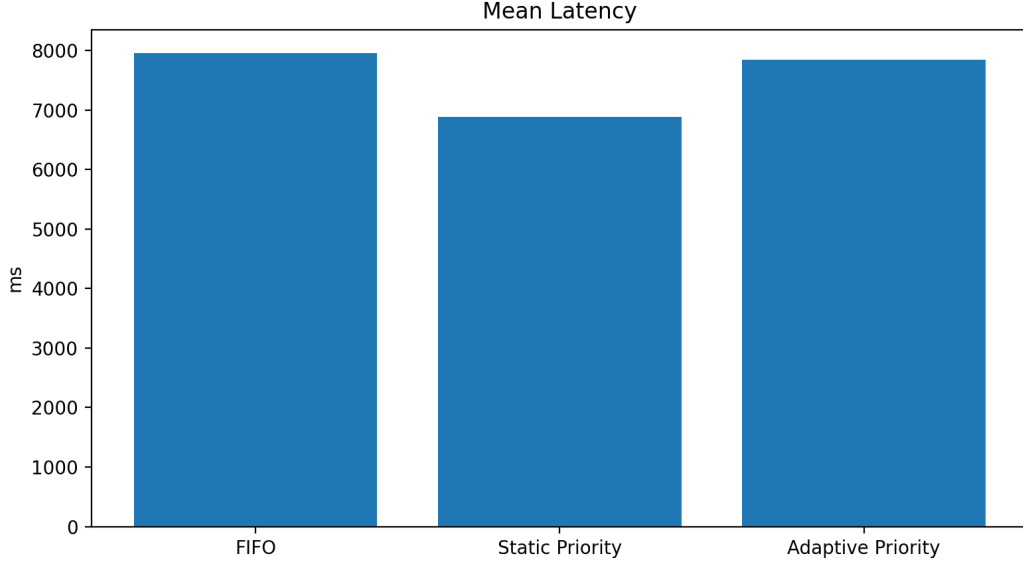


Figure 1: Mean latency comparison across FIFO, static priority, and adaptive priority scheduling. Static priority achieves the lowest average latency, while adaptive priority remains close to FIFO while preserving stronger freshness guarantees.

Figures 2 and 3 show the **P95 and P99 latency** results. Static priority scheduling exhibits significantly worse tail latency compared to the other policies. In our experiments, the P95 latency for static priority exceeded 31,000 ms, and the P99 latency approached 35,000 ms. This behavior occurs because lower-priority updates can be repeatedly delayed by newly arriving higher-priority work, producing starvation-like effects.

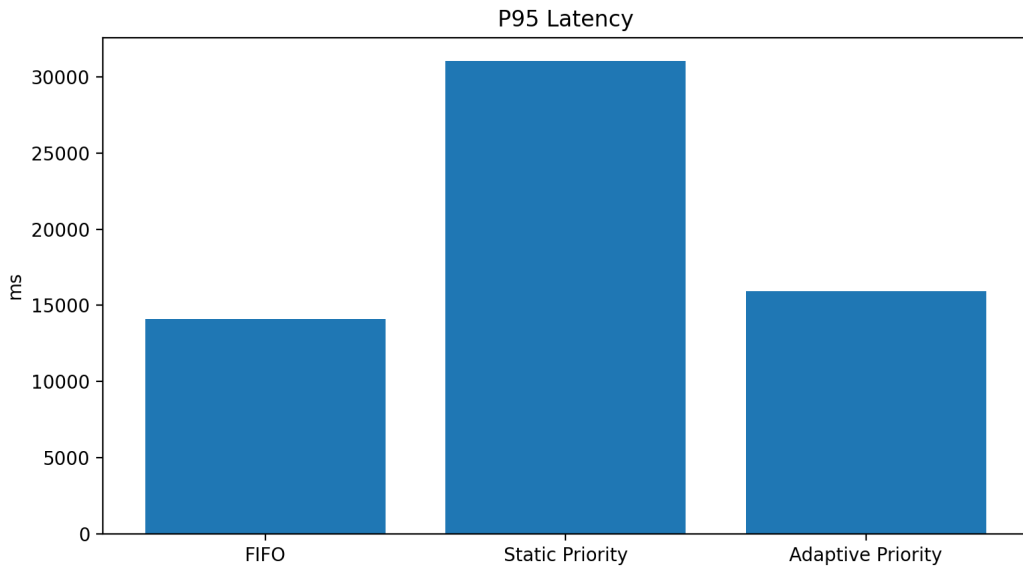


Figure 2: P95 latency comparison across scheduling policies. Static priority exhibits substantially worse tail latency, while adaptive priority reduces extreme delays and remains much closer to FIFO behavior.

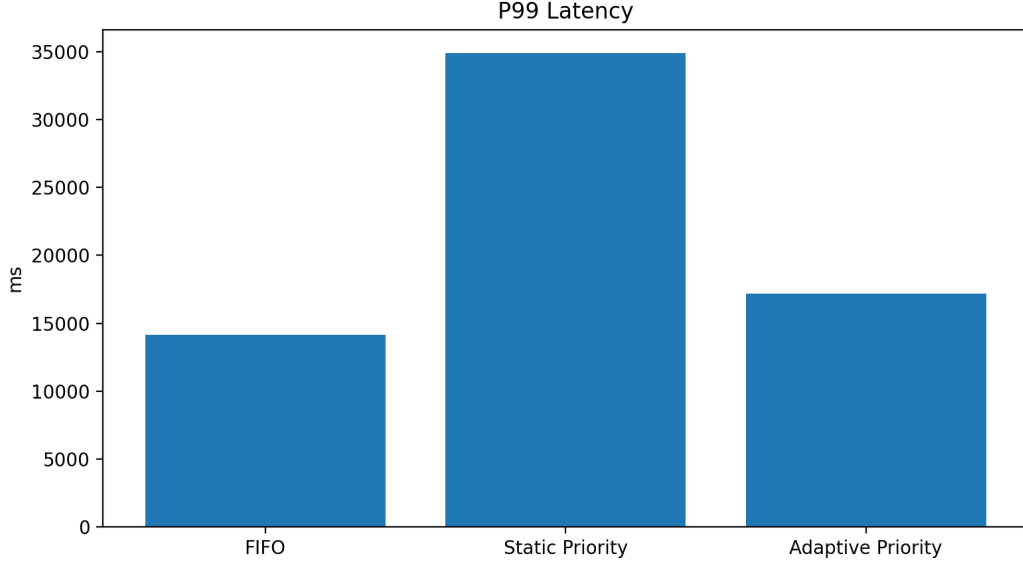


Figure 3: P99 latency comparison across FIFO, static priority, and adaptive priority scheduling. The adaptive scheduler significantly reduces the extreme tail-latency inflation introduced by strict static priority scheduling.

In contrast, both FIFO and the adaptive priority scheduler maintain significantly lower tail latency. The adaptive policy reduces the P95 latency to approximately 15,900 ms and the P99 latency to roughly 17,200 ms. This represents nearly a **50% reduction in tail latency compared to static priority scheduling**, demonstrating that the adaptive approach successfully prevents the extreme delays produced by strict priority scheduling.

7.2 Latency Stability

Figure 4 shows the **standard deviation of latency** for each scheduling policy. Static priority scheduling exhibits the highest latency variance, indicating highly unstable processing times across updates. FIFO scheduling shows the lowest latency variability, as jobs are processed strictly in arrival order.

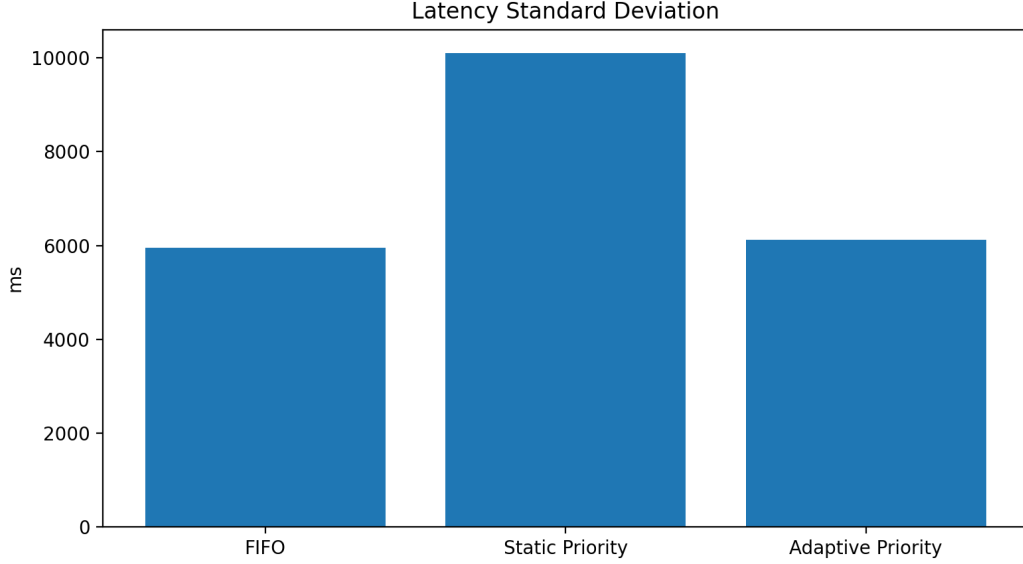


Figure 4: Latency standard deviation across scheduling policies. Static priority produces the greatest latency variability, while adaptive priority remains much closer to FIFO and therefore preserves more stable service behavior.

The adaptive scheduler produces latency variability similar to FIFO while still maintaining improved freshness behavior. This indicates that the adaptive scoring function successfully moderates the aggressive prioritization of fresh updates while preventing starvation of lower-priority streams.

7.3 Freshness Performance

Figure 5 shows the **mean freshness lag** for each scheduling strategy. FIFO scheduling performs poorly on this metric because updates are processed strictly in arrival order regardless of their freshness requirements. As a result, the average freshness lag exceeds 6,900 ms.

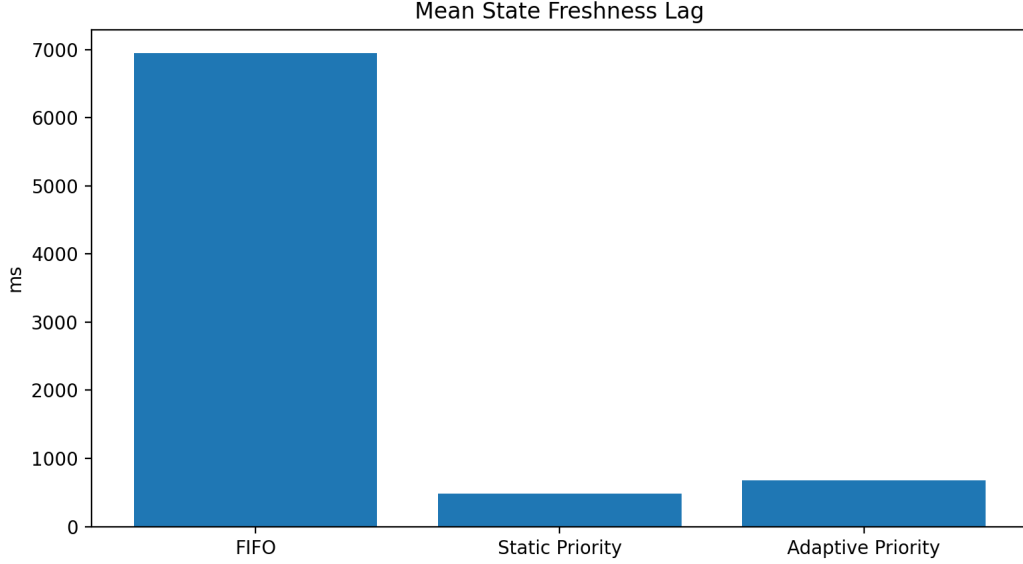


Figure 5: Mean state freshness lag across scheduling policies. FIFO produces the stalest visible state, while static priority minimizes freshness lag. Adaptive priority preserves most of the freshness benefit of static priority while avoiding its worse latency penalties.

Static priority scheduling significantly improves freshness by aggressively prioritizing time-sensitive updates. The mean freshness lag is reduced to approximately 482 ms. The adaptive priority scheduler maintains a mean freshness lag of approximately 677 ms. While this is slightly higher than static priority scheduling, it remains within the same order of magnitude and represents a substantial improvement over FIFO scheduling. This demonstrates that the adaptive scoring approach can preserve most of the freshness benefits of static priority scheduling while avoiding its extreme latency penalties.

7.4 Latency Distribution

Figure 6 presents the **latency cumulative distribution function (CDF)** for each scheduling strategy. The static priority curve exhibits a long heavy tail, confirming the presence of extreme delays for certain updates. FIFO and adaptive scheduling produce significantly tighter latency distributions.

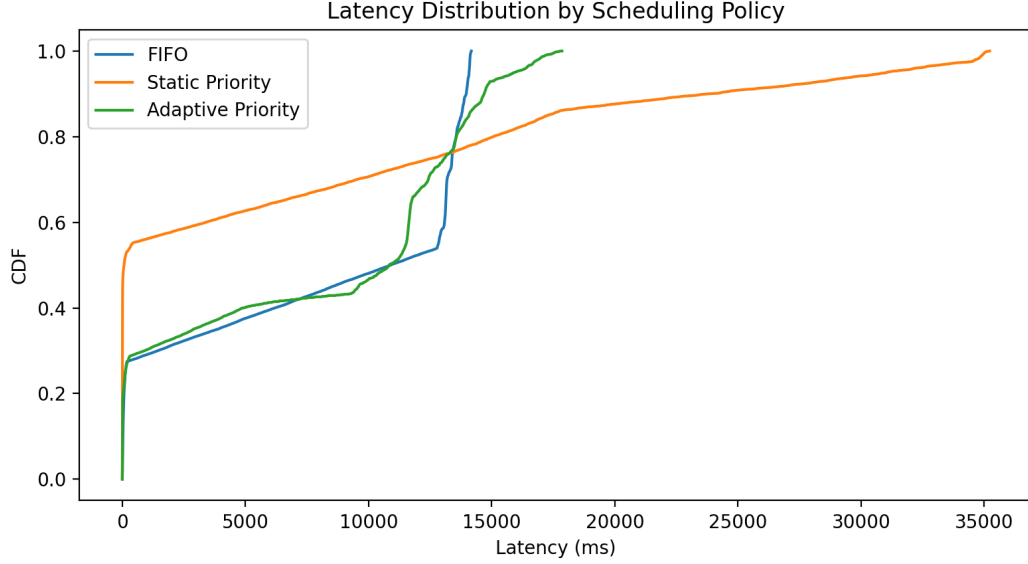


Figure 6: Cumulative distribution of latency under FIFO, static priority, and adaptive priority scheduling. Static priority exhibits a much heavier tail, whereas adaptive priority remains substantially closer to FIFO across most of the distribution while maintaining stronger freshness performance.

The adaptive scheduler closely tracks FIFO performance across most of the latency distribution while maintaining much stronger freshness guarantees. This result demonstrates the key advantage of adaptive arbitration: balancing fairness and freshness without introducing pathological latency behavior.

7.5 Per-Stream Performance

Finally, Figure 7 shows the **P95 latency by stream** under the adaptive scheduler. The results indicate that latency is distributed relatively evenly across burst, control, and background streams. This suggests that the adaptive policy avoids starvation and maintains fairness between workload types while still responding to freshness pressure.

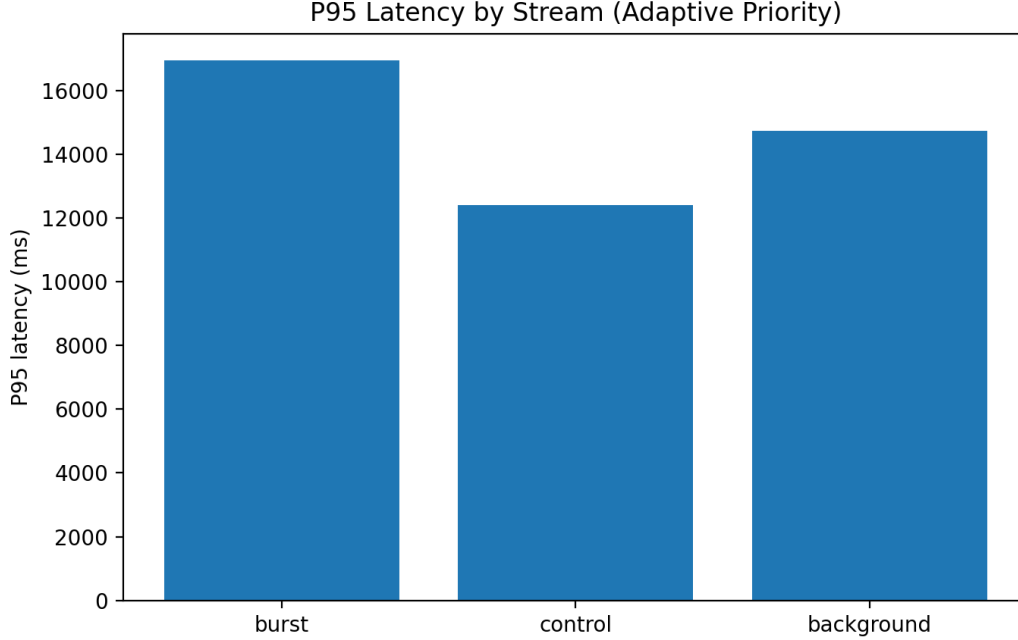


Figure 7: Per-stream P95 latency under adaptive priority scheduling. The adaptive policy maintains relatively balanced tail-latency behavior across burst, control, and background streams, suggesting that it reduces starvation while still responding to freshness pressure.

7.6 Summary

Overall, the results demonstrate that the proposed adaptive priority scheduler effectively balances two competing objectives: minimizing update freshness lag while maintaining stable latency. Compared to static priority scheduling, the adaptive approach significantly reduces tail latency and latency variance while preserving most of the freshness benefits. Compared to FIFO scheduling, the adaptive policy dramatically improves freshness while maintaining similar latency stability.

These results suggest that adaptive arbitration provide a practical compromise between strict fairness and strict priority scheduling for heterogeneous update workloads.

8 Discussion

The experimental results highlight the trade-offs between fairness, latency stability, and state freshness across different scheduling strategies. FIFO scheduling produces the most stable latency distribution but performs poorly with respect to freshness. Because updates are processed strictly in arrival order, time-sensitive updates may remain in the queue while older events are processed first. As a result, FIFO exhibits the largest freshness lag among the evaluated policies. Static priority scheduling demonstrates the opposite behavior. By aggressively prioritizing high-priority of fresh updates, it significantly reduces freshness lag. However, this strict prioritization introduces severe tail-latency inflation. Lower-priority updates may be repeatedly delayed by newly arriving high-priority events, producing starvation-like behavior and extremely high P95 and P99 latency values.

The adaptive priority scheduler attempts to balance these competing objectives by combining multiple signals in a scoring function. In addition to base priority, the scheduler incorporates waiting time, queue backlog, update freshness, and stream variance when selecting the next job to execute. This allows the scheduler to prioritize fresh updates while still accounting for queue pressure and fairness across streams.

The results demonstrate that this adaptive approach successfully moderates the extreme behavior observed in static priority scheduling. Tail latency is significantly reduced while freshness performance remains close to that of static priority. Furthermore, the per-stream results indicate that the adaptive scheduler maintains relatively balanced performance across heterogeneous workloads, suggesting that the scoring function helps prevent starvation while still responding to freshness pressure.

8.1 Limitations

The evaluation presented in this work is based on a simulated workload model rather than a deployed production system. While the simulation captures heterogeneous update streams and varying arrival rates, real-world systems may exhibit additional complexity such as burst amplification, network jitter, or multi-resource contention. Future experiments using real telemetry or streaming workloads would help validate the robustness of the proposed arbitration strategy under operational conditions.

9 Conclusion

This work investigated the trade-offs between latency stability and data freshness in systems that must arbitrate between heterogeneous update streams. Traditional scheduling strategies such as FIFO and static priority represent two extremes: FIFO maintains predictable latency but allows stale updates to accumulate, while strict priority scheduling aggressively favors fresh updates at the cost of severe tail-latency inflation.

We introduced an adaptive priority scheduling approach that combines multiple scheduling signals, including base priority, waiting time, queue backlog, freshness pressure, and per-stream latency variance into a unified scoring function. This design allows the scheduler to dynamically balance fairness, responsiveness, and queue stability without relying on rigid priority rules.

Experimental evaluation demonstrates that the adaptive scheduler significantly moderates the pathological tail behavior observed under static priority scheduling while preserving most of its freshness advantages. In particular, the adaptive policy reduces extreme P95 and P99 latency while maintaining balanced performance across heterogeneous workload streams.

These results suggest that multi-factor adaptive arbitration can provide a practical alternative to fixed scheduling policies in real-time or distributed systems where both latency and freshness are critical. Future work may explore cognitively inspired learning approaches or dynamically tuned weighting strategies that use machine learning or artificial intelligence to adapt arbitration behavior online in response to changing workload conditions. Such approaches could enable schedulers to learn workload structure, predict congestion, and automatically adjust policy trade-offs over time.

References

- [1] Jeffrey Dean and Luiz André Barroso. The Tail at Scale. *Communications of the ACM*, 56(2):74–80, 2013.
- [2] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2011.
- [3] Shiva Davari and Lui Sha. Sources of Unbounded Priority Inversions in Real-Time Systems and Their Solutions. *ACM SIGPLAN Notices*, 27(9):110–119, 1992.
- [4] Matei Zaharia, Dhruba Borthakur, Joydeep Sen Sarma, Kannan Muthukkaruppan, Scott Shenker, and Ion Stoica. Delay Scheduling: A Simple Technique for Achieving Locality and Fairness in Cluster Scheduling. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*, 2010.
- [5] C. L. Liu and James W. Layland. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, 20(1):46–61, 1973.
- [6] Paris Carbone, Gyula Fóra, Stephan Ewen, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin*, 38(4):28–38, 2015.